

C++友元、异常和高级特性完整自学手册

目录

1. 友元概念详解
 2. 友元类详细学习
 3. 友元成员函数
 4. 嵌套类深入理解
 5. 异常处理机制
 6. RTTI运行时类型识别
 7. 类型转换运算符
 8. 实践项目
 9. 常见问题解答
-

友元概念详解

什么是友元？

想象一下现实生活中的友情关系：你可能会给最好的朋友你家的钥匙，让他们可以进入你的私人空间。在C++中，友元就是这样的概念——类可以授权其他函数或类访问自己的私有成员。

核心理念：

- 友元是单向的（A是B的朋友，不意味着B是A的朋友）
- 友元不能被继承
- 友元关系不具有传递性
- 友元是类的设计者主动授予的权限

为什么需要友元？

考虑这个场景：你正在设计一个银行账户类，需要一个全局函数来处理两个账户之间的转账。这个函数需要访问两个账户的私有余额信息，但你又不想把余额设为公有成员。友元就是解决这个问题的最佳方案。

```
// 示例：银行转账场景
class BankAccount {
private:
    double balance;
    string accountNumber;

public:
    BankAccount(string accNum, double initialBalance)
        : accountNumber(accNum), balance(initialBalance) {}

    // 声明转账函数为友元
    friend bool transfer(BankAccount& from, BankAccount& to, double amount);

    void showBalance() const {
        cout << "账户 " << accountNumber << " 余额: $" << balance << endl;
    }
};

// 友元函数实现
bool transfer(BankAccount& from, BankAccount& to, double amount) {
    if (from.balance >= amount) { // 可以直接访问私有成员
        from.balance -= amount;
        to.balance += amount;
        cout << "转账成功: $" << amount << endl;
        return true;
    }
    cout << "余额不足，转账失败" << endl;
    return false;
}
```

友元类详细学习

友元类的概念

当一个类需要频繁访问另一个类的私有成员时，可以将整个类声明为友元类。这意味着友元类的所有成员函数都可以访问原始类的私有和保护成员。

经典示例：电视机和遥控器

让我们通过一个直观的例子来理解友元类——电视机和遥控器的关系：

CPP

```
#include <iostream>
using namespace std;

class Tv {
public:
    // 声明Remote类为友元类
    friend class Remote;

    // 电视机状态枚举
    enum {Off, On};
    enum {MinVol, MaxVol = 20};
    enum {Antenna, Cable};
    enum {TV, DVD};

private:
    int state;    // 开关状态
    int volume;   // 音量
    int maxchannel; // 最大频道数
    int channel;  // 当前频道
    int mode;     // 信号模式 (天线/有线)
    int input;    // 输入源 (TV/DVD)

public:
    // 构造函数
    Tv(int s = Off, int mc = 125)
        : state(s), volume(5), maxchannel(mc),
          channel(2), mode(Cable), input(TV) {}

    // 电视机本身的控制方法
    void onoff() { state = (state == On) ? Off : On; }
    bool ison() const { return state == On; }

    bool volup() {
        if (volume < MaxVol) {
            volume++;
            return true;
        }
        return false;
    }

    bool voldown() {
        if (volume > MinVol) {
            volume--;
            return true;
        }
        return false;
    }
}
```

```

}

void chanup() {
    if (channel < maxchannel)
        channel++;
    else
        channel = 1;
}

void chandown() {
    if (channel > 1)
        channel--;
    else
        channel = maxchannel;
}

void set_mode() { mode = (mode == Antenna ? Cable : Antenna; }
void set_input() { input = (input == TV ? DVD : TV; }

void settings() const {
    cout << "TV状态: " << (state == Off ? "关闭" : "开启") << endl;
    if (state == On) {
        cout << "音量设置: " << volume << endl;
        cout << "频道设置: " << channel << endl;
        cout << "模式: " << (mode == Antenna ? "天线" : "有线") << endl;
        cout << "输入: " << (input == TV ? "TV" : "DVD") << endl;
    }
}

};

class Remote {
private:
    int mode; // 遥控器模式

public:
    Remote(int m = Tv::TV) : mode(m) {}

    // 遥控器方法可以直接访问Tv的私有成员
    bool volup(Tv& t) { return t.volup(); }
    bool voldown(Tv& t) { return t.voldown(); }
    void onoff(Tv& t) { t.onoff(); }
    void chanup(Tv& t) { t.chanup(); }
    void chandown(Tv& t) { t.chandown(); }

    // 直接设置频道 (遥控器特有功能)
    void set_chan(Tv& t, int c) {
        if (c >= 1 && c <= t.maxchannel) {

```

```

        t.channel = c; // 直接访问私有成员
    }
}

void set_mode(Tv& t) { t.set_mode(); }
void set_input(Tv& t) { t.set_input(); }
};

// 测试程序
int main() {
    Tv myTV;
    Remote myRemote;

    cout << "=== 初始设置 ===" << endl;
    myTV.settings();

    cout << "\n=== 使用遥控器操作 ===" << endl;
    myRemote.onoff(myTV);    // 开机
    myRemote.set_chan(myTV, 10); // 设置频道为10
    myRemote.volup(myTV);    // 增加音量
    myRemote.volup(myTV);    // 再次增加音量

    cout << "操作后的设置:" << endl;
    myTV.settings();

    return 0;
}

```

友元类的设计要点

何时使用友元类：

1. 两个类之间有密切的协作关系
2. 一个类需要频繁访问另一个类的私有成员
3. 设计模式中的某些情况（如迭代器模式）

注意事项：

- 友元破坏了封装性，应谨慎使用
- 友元关系不是相互的
- 友元声明可以放在类的任何部分（public、private、protected）

友元成员函数

更精确的友元控制

有时候，我们不需要整个类都成为友元，只需要某个特定的成员函数能够访问私有成员。这时可以使用友元成员函数。

实现步骤和注意事项

友元成员函数的声明顺序很重要：

```
cpp
```

```
#include <iostream>
using namespace std;

// 第一步：前向声明
class Tv;

// 第二步：定义Remote类（只有方法声明）
class Remote {
private:
    int mode;

public:
    Remote(int m = 0) : mode(m) {}

    // 只声明，不定义
    bool volup(Tv& t);
    bool voldown(Tv& t);
    void onoff(Tv& t);
    void chanup(Tv& t);
    void chandown(Tv& t);
    void set_mode(Tv& t);
    void set_input(Tv& t);
    void set_chan(Tv& t, int c); // 这个方法将成为友元
};

// 第三步：定义Tv类
class Tv {
public:
    // 只声明特定的成员函数为友元
    friend void Remote::set_chan(Tv& t, int c);

    enum {Off, On};
    enum {MinVol, MaxVol = 20};
    enum {Antenna, Cable};
    enum {TV, DVD};

private:
    int state;
    int volume;
    int maxchannel;
    int channel;
    int mode;
    int input;

public:
    Tv(int s = Off, int mc = 125)
```

```

: state(s), volume(5), maxchannel(mc),
  channel(2), mode(Cable), input(TV) {}

void onoff() { state = (state == On) ? Off : On; }
bool ison() const { return state == On; }

bool volup() {
    if (volume < MaxVol) {
        volume++;
        return true;
    }
    return false;
}

bool voldown() {
    if (volume > MinVol) {
        volume--;
        return true;
    }
    return false;
}

void chanup() {
    if (channel < maxchannel)
        channel++;
    else
        channel = 1;
}

void chandown() {
    if (channel > 1)
        channel--;
    else
        channel = maxchannel;
}

void set_mode() { mode = (mode == Antenna) ? Cable : Antenna; }
void set_input() { input = (input == TV) ? DVD : TV; }

void settings() const {
    cout << "TV状态: " << (state == Off ? "关闭" : "开启") << endl;
    if (state == On) {
        cout << "音量: " << volume << ", 频道: " << channel << endl;
        cout << "模式: " << (mode == Antenna ? "天线" : "有线") << endl;
        cout << "输入: " << (input == TV ? "TV" : "DVD") << endl;
    }
}
}

```



```
};

// 第四步：实现Remote类的方法
inline bool Remote::volup(Tv& t) { return t.volup(); }
inline bool Remote::voldown(Tv& t) { return t.voldown(); }
inline void Remote::onoff(Tv& t) { t.onoff(); }
inline void Remote::chanup(Tv& t) { t.chanup(); }
inline void Remote::chardown(Tv& t) { t.chardown(); }
inline void Remote::set_mode(Tv& t) { t.set_mode(); }
inline void Remote::set_input(Tv& t) { t.set_input(); }

// 友元成员函数可以直接访问私有成员
inline void Remote::set_chan(Tv& t, int c) {
    if (c >= 1 && c <= t.maxchannel) {
        t.channel = c; // 直接访问私有成员
    }
}
```

声明顺序的重要性

理解为什么需要这样的顺序：

1. **前向声明Tv**：让编译器知道Tv是一个类名
2. **声明Remote类**：此时只能使用Tv的指针或引用，不能使用具体成员
3. **定义Tv类**：包含友元声明
4. **实现Remote方法**：此时编译器已经知道Tv的完整定义

嵌套类深入理解

嵌套类的概念

嵌套类是在另一个类内部定义的类。它主要用于创建仅为外部类服务的辅助类，避免名称污染。

实际应用：改进的队列类

让我们通过一个队列类的例子来学习嵌套类：

CPP

```
#include <iostream>
#include <string>
using namespace std;

template<class Item>
class Queue {
private:
    enum {Q_SIZE = 10};

    // 嵌套类定义
    class Node {
    public:
        Item item;
        Node* next;

        // 嵌套类的构造函数
        Node(const Item& i) : item(i), next(nullptr) {
            cout << "创建节点: " << item << endl;
        }

        ~Node() {
            cout << "销毁节点: " << item << endl;
        }
    };

    Node* front; // 队列前端
    Node* rear;  // 队列后端
    int items;   // 当前项目数
    const int qsize; // 队列最大容量

    // 禁用拷贝构造和赋值 (简化示例)
    Queue(const Queue& q) : qsize(0) {}
    Queue& operator=(const Queue& q) { return *this; }

public:
    Queue(int qs = Q_SIZE) : qsize(qs) {
        front = rear = nullptr;
        items = 0;
        cout << "创建队列, 容量: " << qsize << endl;
    }

    ~Queue() {
        cout << "开始销毁队列..." << endl;
        Node* temp;
        while (front != nullptr) {
            temp = front;
```

```
    front = front->next;
    delete temp;
}
cout << "队列销毁完成" << endl;
}
```

```
bool isempty() const { return items == 0; }
bool isfull() const { return items == qsize; }
int queuecount() const { return items; }
```

// 入队

```
bool enqueue(const Item& item) {
    if (isfull()) {
        cout << "队列已满，无法添加：" << item << endl;
        return false;
    }
```

```
    Node* add = new Node(item); // 使用嵌套类创建节点
    items++;
```

```
    if (front == nullptr) { // 队列为空
        front = add;
    } else {
        rear->next = add;
    }
    rear = add;

    cout << "成功入队：" << item << " (当前大小：" << items << ")" << endl;
    return true;
}
```

// 出队

```
bool dequeue(Item& item) {
    if (front == nullptr) {
        cout << "队列为空，无法出队" << endl;
        return false;
    }
```

```
    item = front->item;
    items--;
    Node* temp = front;
    front = front->next;
```

```
    if (items == 0) {
        rear = nullptr;
    }
```

```
    cout << "成功出队: " << item << " (当前大小: " << items << ")" << endl;
    delete temp;
    return true;
}

// 显示队列内容
void display() const {
    if (isempty()) {
        cout << "队列为空" << endl;
        return;
    }

    cout << "队列内容: ";
    Node* temp = front;
    while (temp != nullptr) {
        cout << temp->item << " ";
        temp = temp->next;
    }
    cout << endl;
}

};

// 测试程序
int main() {
    cout << "=== 字符串队列测试 ===" << endl;
    Queue<string> names(5);

    // 入队测试
    names.enqueue("Alice");
    names.enqueue("Bob");
    names.enqueue("Charlie");
    names.display();

    // 出队测试
    string name;
    names.dequeue(name);
    cout << "出队的元素: " << name << endl;
    names.display();

    // 继续添加元素
    names.enqueue("David");
    names.enqueue("Eve");
    names.enqueue("Frank");
    names.display();

    // 测试队列满的情况
    names.enqueue("Grace"); // 应该失败
```

```
cout << "\n=== 整数队列测试 ===" << endl;
Queue<int> numbers(3);

for (int i = 1; i <= 4; i++) {
    numbers.enqueue(i * 10);
}

numbers.display();

return 0;
}
```

嵌套类的访问权限

嵌套类的访问权限遵循以下规则：

CPP

```
class Outer {
private:
    class PrivateNested {
        // 只有Outer类可以使用
    };

protected:
    class ProtectedNested {
        // Outer类和其派生类可以使用
    };

public:
    class PublicNested {
        // 任何地方都可以使用，但需要Outer::PublicNested
    };

    void demonstrateAccess() {
        PrivateNested pn; // OK
        ProtectedNested prn; // OK
        PublicNested pub; // OK
    }
};

// 在类外部
void testAccess() {
    // Outer::PrivateNested pn; // 错误! 不可访问
    // Outer::ProtectedNested prn; // 错误! 不可访问
    Outer::PublicNested pub; // OK
}
```

异常处理机制

异常处理的必要性

在程序运行过程中，总会遇到各种意外情况：文件打不开、内存不足、网络连接失败、用户输入错误数据等。传统的错误处理方法（返回错误码、全局错误变量）有很多缺点，而异常处理提供了一种更优雅、更安全的解决方案。

异常处理的基本语法

异常处理包含三个关键字：`try` `throw` `catch`

cpp

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

// 自定义异常类
class DivisionByZeroException {
private:
    string message;

public:
    DivisionByZeroException(const string& msg) : message(msg) {}

    const string& what() const {
        return message;
    }
};

// 安全的除法函数
double safeDivide(double a, double b) {
    if (b == 0.0) {
        throw DivisionByZeroException("除数不能为零！");
    }
    return a / b;
}

// 演示基本异常处理
void basicExceptionDemo() {
    cout << "=== 基本异常处理演示 ===" << endl;

    double x, y;
    cout << "请输入两个数字（被除数 除数）：";
    cin >> x >> y;

    try {
        double result = safeDivide(x, y);
        cout << x << " / " << y << " = " << result << endl;
    }
    catch (const DivisionByZeroException& e) {
        cout << "捕获到异常: " << e.what() << endl;
        cout << "程序继续执行..." << endl;
    }
}

int main() {
    basicExceptionDemo();
}
```

```
return 0;  
}
```

异常类层次结构设计

设计一个实用的异常类层次结构：

CPP


```
#include <iostream>
#include <stdexcept>
#include <string>
#include <cmath>
using namespace std;

// 基础数学异常类
class MathException : public logic_error {
protected:
    double value1, value2;

public:
    MathException(const string& msg, double v1 = 0, double v2 = 0)
        : logic_error(msg), value1(v1), value2(v2) {}

    virtual void showDetails() const {
        cout << "数学异常: " << what() << endl;
        cout << "相关值: " << value1 << ", " << value2 << endl;
    }

    double getValue1() const { return value1; }
    double getValue2() const { return value2; }
};

// 除零异常
class DivideByZeroException : public MathException {
public:
    DivideByZeroException(double dividend, double divisor)
        : MathException("除零错误", dividend, divisor) {}

    void showDetails() const override {
        cout << "除零异常: 尝试用 " << value2 << " 除 " << value1 << endl;
        cout << "除数不能为零! " << endl;
    }
};

// 负数平方根异常
class NegativeSqrtException : public MathException {
public:
    NegativeSqrtException(double value)
        : MathException("负数平方根错误", value) {}

    void showDetails() const override {
        cout << "负数平方根异常: 尝试对 " << value1 << " 求平方根" << endl;
        cout << "不能对负数求平方根! " << endl;
    }
}
```

```
};
```

```
// 数学运算器类
```

```
class MathCalculator {
```

```
public:
```

```
    static double divide(double a, double b) {
```

```
        if (b == 0.0) {
```

```
            throw DivideByZeroException(a, b);
```

```
        }
```

```
        return a / b;
```

```
    }
```

```
    static double safeSqrt(double x) {
```

```
        if (x < 0) {
```

```
            throw NegativeSqrtException(x);
```

```
        }
```

```
        return sqrt(x);
```

```
    }
```

```
    static double harmonicMean(double a, double b) {
```

```
        if (a == -b) {
```

```
            throw MathException("调和平均数计算错误: a = -b", a, b);
```

```
        }
```

```
        return 2.0 * a * b / (a + b);
```

```
    }
```

```
    static double geometricMean(double a, double b) {
```

```
        if (a < 0 || b < 0) {
```

```
            throw MathException("几何平均数计算错误: 参数不能为负", a, b);
```

```
        }
```

```
        return sqrt(a * b);
```

```
    }
```

```
};
```

```
// 交互式计算器
```

```
void interactiveCalculator() {
```

```
    cout << "=== 交互式数学计算器 ===" << endl;
```

```
    cout << "选择操作: " << endl;
```

```
    cout << "1. 除法" << endl;
```

```
    cout << "2. 平方根" << endl;
```

```
    cout << "3. 调和平均数" << endl;
```

```
    cout << "4. 几何平均数" << endl;
```

```
    cout << "0. 退出" << endl;
```

```
    int choice;
```

```
    double a, b;
```

```
while (true) {
    cout << "\n请选择操作 (0-4): ";
    cin >> choice;

    if (choice == 0) break;

    try {
        switch (choice) {
            case 1:
                cout << "输入被除数和除数: ";
                cin >> a >> b;
                cout << "结果: " << MathCalculator::divide(a, b) << endl;
                break;

            case 2:
                cout << "输入要求平方根的数: ";
                cin >> a;
                cout << "结果: " << MathCalculator::safeSqrt(a) << endl;
                break;

            case 3:
                cout << "输入两个数计算调和平均数: ";
                cin >> a >> b;
                cout << "结果: " << MathCalculator::harmonicMean(a, b) << endl;
                break;

            case 4:
                cout << "输入两个数计算几何平均数: ";
                cin >> a >> b;
                cout << "结果: " << MathCalculator::geometricMean(a, b) << endl;
                break;

            default:
                cout << "无效选择! " << endl;
        }
    }

    catch (const DivideByZeroException& e) {
        e.showDetails();
        cout << "提示: 请重新输入非零除数" << endl;
    }

    catch (const NegativeSqrtException& e) {
        e.showDetails();
        cout << "提示: 请输入非负数" << endl;
    }

    catch (const MathException& e) {
        e.showDetails();
        cout << "提示: 请检查输入参数" << endl;
    }
}
```

```
    }  
    catch (const exception& e) {  
        cout << "未知异常: " << e.what() << endl;  
    }  
}  
  
cout << "感谢使用计算器! " << endl;  
}  
  
int main() {  
    interactiveCalculator();  
    return 0;  
}
```

栈解退机制

了解异常如何影响对象的生命周期：

CPP

```
#include <iostream>
#include <string>
using namespace std;

// 用于演示栈解退的类
class Demo {
private:
    string name;

public:
    Demo(const string& n) : name(n) {
        cout << "创建 Demo 对象: " << name << endl;
    }

    ~Demo() {
        cout << "销毁 Demo 对象: " << name << endl;
    }

    void show() const {
        cout << "Demo 对象 " << name << " 正在运行" << endl;
    }
};

// 可能抛出异常的函数
void riskyFunction(int level) {
    Demo localObj("Level" + to_string(level));
    localObj.show();

    if (level == 2) {
        throw runtime_error("在Level 2发生异常! ");
    }

    cout << "Level " << level << " 正常结束" << endl;
}

void middleFunction() {
    Demo middleObj("Middle");
    middleObj.show();

    cout << "调用 riskyFunction..." << endl;
    riskyFunction(2); // 这里会抛出异常

    cout << "这行代码不会执行" << endl; // 不会执行
}

void demonstrateStackUnwinding() {
```

```
cout << "=== 栈解退演示 ===" << endl;

Demo mainObj("Main");

try {
    mainObj.show();
    cout << "调用 middleFunction..." << endl;
    middleFunction();
    cout << "这行代码不会执行" << endl; // 不会执行
}
catch (const runtime_error& e) {
    cout << "捕获异常: " << e.what() << endl;
    cout << "注意观察对象的销毁顺序" << endl;
}

mainObj.show();
cout << "程序正常结束" << endl;
}

int main() {
    demonstrateStackUnwinding();
    return 0;
}
```

RTTI运行时类型识别

RTTI的应用场景

RTTI（运行时类型识别）允许程序在运行时确定对象的实际类型。这在处理多态对象时特别有用。

dynamic_cast详解

CPP

```
#include <iostream>
#include <vector>
#include <memory>
#include <typeinfo>
using namespace std;

// 基类
class Animal {
public:
    virtual ~Animal() = default; // 虚析构函数, 启用RTTI
    virtual void makeSound() const = 0;
    virtual void move() const {
        cout << "动物在移动" << endl;
    }
};

// 派生类
class Dog : public Animal {
private:
    string breed;

public:
    Dog(const string& b) : breed(b) {}

    void makeSound() const override {
        cout << "汪汪! 我是一只" << breed << endl;
    }

    void fetch() const {
        cout << breed << "正在捡球" << endl;
    }
};
```